MON APPRENTISSAGE — SERIES PYTHON

Python

Du zéro à l'expert

Chapitre 14 — Les Modules et Packages

Importer le monde — tirer parti de l'écosystème Python

Auteur :

KETOTSA AMÉVI CLAUDE

Date de publication :

12 mars 2026

Publié dans le cadre de mon parcours d'apprentissage Python

Table des matières

Chapitre 14 — Les Modules et Packages **2**
0.1 Importer un module — import 2
0.2 Le module math — Mathématiques 3
0.3 Le module random — Aléatoire 4
0.4 Le module os — Système d'exploitation 5
0.5 Le module sys — Système Python 5
0.6 Le module datetime — Dates et heures 6
0.7 Créer ses propres modules 7
0.8 La variable __name__ 8
0.9 Les packages — Organiser ses modules 9
0.10 Les modules standards incontournables 9
0.11 Ce qui m'a le plus surpris dans ce chapitre 10
0.12 Résumé du Chapitre 14 10

Chapitre 14 — Les Modules et Packages

À retenir

Un **module** est un fichier Python contenant des fonctions, des classes et des variables réutilisables. Un **package** est un dossier de modules organisés ensemble. Python est livré avec une **bibliothèque standard** extraordinairement riche — des centaines de modules prêts à l'emploi. Savoir les importer et les utiliser, c'est multiplier sa puissance par mille sans écrire une seule ligne de plus.

Importer un module — import

Les différentes formes d'import

```python
# Forme 1      import complet
import math
print(math.pi)           # 3.141592653589793
print(math.sqrt(16))     # 4.0
print(math.ceil(4.2))    # 5
print(math.floor(4.8))   # 4

# Forme 2      import avec alias
import math as m
print(m.pi)          # 3.141592653589793
print(m.sqrt(25))    # 5.0

# Forme 3      import selectif
from math import pi, sqrt, factorial
print(pi)            # 3.141592653589793
print(sqrt(144))     # 12.0
print(factorial(5))  # 120

# Forme 4      import selectif avec alias
from math import sqrt as racine_carree
print(racine_carree(81))    # 9.0

#  Forme deconseillee      import tout (*)
# from math import *        pollue l'espace de noms
```

Erreur fréquente

`from module import *` importe **tout** dans l'espace de noms courant — y compris des noms qui pourraient écraser vos propres variables. C'est une pratique dangereuse dans les grands projets. Toujours préférer un import explicite et ciblé.

Le module math — Mathématiques

module math — fonctions essentielles

```python
import math

# Constantes
print(math.pi)     # 3.141592653589793
print(math.e)      # 2.718281828459045
print(math.tau)    # 6.283185307179586  (2*pi)
print(math.inf)    # inf
print(math.nan)    # nan

# Arrondis
print(math.ceil(4.1))     # 5   (arrondi superieur)
print(math.floor(4.9))    # 4   (arrondi inferieur)
print(math.trunc(4.9))    # 4   (partie entiere)
print(round(4.567, 2))    # 4.57

# Puissances et racines
print(math.sqrt(64))       # 8.0
print(math.pow(2, 10))     # 1024.0
print(math.log(100, 10))   # 2.0  (log base 10)
print(math.log2(1024))     # 10.0
print(math.log(math.e))    # 1.0  (log naturel)

# Trigonometrie (en radians)
print(math.sin(math.pi/2))    # 1.0
print(math.cos(0))             # 1.0
print(math.degrees(math.pi))   # 180.0
print(math.radians(180))       # 3.141592...

# Valeur absolue et signe
print(math.fabs(-4.5))    # 4.5
print(math.gcd(48, 18))   # 6  (PGCD)

# Cas pratique : calculer l'hypotenuse
a, b = 3, 4
hypo = math.sqrt(a**2 + b**2)
print(f"Hypotenuse : {hypo}")    # 5.0
# Ou encore plus direct :
print(math.hypot(a, b))          # 5.0
```

Le module random — Aléatoire

module random — nombres aléatoires

```python
import random

# Nombre flottant entre 0 et 1
print(random.random())          # ex: 0.7432...

# Entier entre a et b (inclus)
print(random.randint(1, 6))     # de a 6 faces

# Flottant entre a et b
print(random.uniform(1.0, 10.0))

# Choisir un element aleatoire
fruits = ["pomme", "banane", "cerise", "mangue"]
print(random.choice(fruits))

# Choisir k elements (sans repetition)
print(random.sample(fruits, 2))

# Choisir k elements (avec repetition)
print(random.choices(fruits, k=3))

# M langer une liste en place
nombres = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
random.shuffle(nombres)
print(nombres)

# Graine     reproduire les memes resultats
random.seed(42)
print(random.randint(1, 100))   # toujours 52 avec seed=42
print(random.randint(1, 100))   # toujours 8

# Cas pratique : simuler un lancer de des
def lancer_des(nb_des=2, nb_faces=6):
    resultats = [random.randint(1, nb_faces) for _ in range(
        nb_des)]
    return resultats, sum(resultats)

des, total = lancer_des()
print(f"D  s  : {des}     Total : {total}")
```

Le module os — Système d'exploitation

module os — interagir avec le système

```python
import os

# Repertoire courant
print(os.getcwd())    # /home/utilisateur/projets

# Lister les fichiers d'un dossier
fichiers = os.listdir(".")
print(fichiers)

# Creer et supprimer des dossiers
os.makedirs("mon_projet/data", exist_ok=True)
# os.rmdir("dossier_vide")

# Verifier l'existence
print(os.path.exists("mon_fichier.txt"))    # True/False
print(os.path.isfile("script.py"))          # True/False
print(os.path.isdir("mon_projet"))          # True/False

# Manipuler les chemins (portable Windows/Linux/Mac)
chemin = os.path.join("dossier", "sous_dossier", "fichier.txt")
print(chemin)    # dossier/sous_dossier/fichier.txt (Linux)

# Decomposer un chemin
chemin_complet = "/home/alice/projets/script.py"
print(os.path.dirname(chemin_complet))     # /home/alice/projets
print(os.path.basename(chemin_complet))    # script.py
nom, ext = os.path.splitext("script.py")
print(nom, ext)    # script  .py

# Variables d'environnement
python_path = os.environ.get("PATH", "Non defini")
print(f"PATH : {python_path[:50]}...")

# Taille d'un fichier
taille = os.path.getsize("script.py")
print(f"Taille : {taille} octets")
```

Le module sys — Système Python

module sys — informations système

```python
import sys

# Version de Python
print(sys.version)
```

```python
print(sys.version_info)
# sys.version_info(major=3, minor=11, ...)

# Plateforme
print(sys.platform)   # 'linux', 'win32', 'darwin'

# Arguments de la ligne de commande
# python script.py arg1 arg2
print(sys.argv)   # ['script.py', 'arg1', 'arg2']

# Chemin de recherche des modules
for chemin in sys.path[:3]:
    print(chemin)

# Quitter le programme
# sys.exit(0)    # 0 = succes, autre = erreur

# Taille d'un objet en memoire
liste = [1, 2, 3, 4, 5]
print(sys.getsizeof(liste))   # taille en octets

# Encodage par defaut
print(sys.getdefaultencoding())   # utf-8

# Flux standard
sys.stdout.write("Bonjour sans saut de ligne")
sys.stderr.write("Message d'erreur\n")
```

Le module datetime — Dates et heures

module datetime — manipuler les dates

```python
from datetime import datetime, date, time, timedelta

# Date et heure actuelles
maintenant = datetime.now()
print(maintenant)
# 2024-03-15 14:32:45.123456

# Extraire les composantes
print(maintenant.year)     # 2024
print(maintenant.month)    # 3
print(maintenant.day)      # 15
print(maintenant.hour)     # 14
print(maintenant.minute)   # 32

# Creer une date specifique
naissance = date(1999, 6, 15)
```

```python
print(naissance)    # 1999-06-15

# Calculer un age
aujourd_hui = date.today()
age = aujourd_hui.year - naissance.year
print(f"Age : {age} ans")

# Arithmetique de dates avec timedelta
dans_30_jours = aujourd_hui + timedelta(days=30)
print(f"Dans 30 jours : {dans_30_jours}")

il_y_a_une_semaine = aujourd_hui - timedelta(weeks=1)
print(f"Il y a 7 jours : {il_y_a_une_semaine}")

# Difference entre deux dates
debut = date(2024, 1, 1)
fin = date(2024, 12, 31)
duree = fin - debut
print(f"Jours dans 2024 : {duree.days}")

# Formater une date en chaine
print(maintenant.strftime("%d/%m/%Y    %H:%M"))
#     15/03/2024    14:32

# Parser une chaine en date
chaine = "25/12/2024"
noel = datetime.strptime(chaine, "%d/%m/%Y")
print(noel)
```

Créer ses propres modules

Créer et utiliser son propre module

```python
# Fichier : utilitaires.py
# ========================
def saluer(prenom):
    """Salue une personne."""
    return f"Bonjour, {prenom} !"

def calculer_moyenne(notes):
    """Calcule la moyenne d'une liste."""
    if not notes:
        return 0
    return sum(notes) / len(notes)

PI = 3.14159
VERSION = "1.0.0"
```

```python
# Fichier : main.py (dans le meme dossier)
# ========================================
import utilitaires

print(utilitaires.saluer("Alice"))
print(utilitaires.calculer_moyenne([15, 18, 12]))
print(utilitaires.PI)
print(utilitaires.VERSION)

# Import selectif depuis son module
from utilitaires import saluer, calculer_moyenne

print(saluer("Bob"))
print(calculer_moyenne([9, 14, 17, 11]))
```

La variable __name__

</> $_{name_moduleprincipalouimporté?}$

```python
# Fichier : mon_module.py
# =======================

def ma_fonction():
    return "Je suis une fonction du module"

def tester():
    print("Tests en cours...")
    print(ma_fonction())
    print("Tests r ussis !")

# Ce bloc ne s'execute QUE si on lance ce fichier directement
# Il est IGNOR  si le fichier est importe par un autre
if __name__ == "__main__":
    print("Execution directe d tectee.")
    tester()

# Quand on importe : __name__ == "mon_module"
# Quand on execute directement : __name__ == "__main__"
```

À retenir

`if __name__ == "__main__"` est un pattern fondamental en Python. Il permet à un fichier d'être à la fois un **module importable** et un **script exécutable**. Le code dans ce bloc sert aux tests et à l'utilisation directe — il est ignoré lors des imports. C'est une bonne pratique systématique dans tout module Python.

Les packages — Organiser ses modules

Structure d'un package Python

```
% Structure de fichiers
% mon_projet/
%   __init__.py              fait du dossier un package
%   utilitaires.py
%   mathematiques/
%       __init__.py
%       geometrie.py
%       statistiques.py
%   donnees/
%       __init__.py
%       lecture.py
%       ecriture.py

% Contenu de mon_projet/__init__.py
VERSION = "2.0.0"
NOM     = "MonProjet"

% Contenu de mon_projet/mathematiques/geometrie.py
import math

def aire_cercle(r):
    return math.pi * r ** 2

def aire_rectangle(l, L):
    return l * L

% Utilisation depuis main.py
from mon_projet.mathematiques.geometrie import aire_cercle
from mon_projet import VERSION

print(f"Version : {VERSION}")
print(f"Aire : {aire_cercle(5):.2f}")
```

Les modules standards incontournables

os Système de fichiers	**sys** Système Python	**math** Mathématiques
random Aléatoire	**datetime** Dates et heures	**json** Données JSON
re Expressions régulières	**collections** Structures avancées	**itertools** Itérateurs avancés
pathlib Chemins modernes	**time** Temps et pauses	**string** Constantes texte

Ce qui m'a le plus surpris dans ce chapitre

Ce qui m'a résisté

Trois révélations de ce chapitre :

1. `__name__ == "__main__"` — un garde-fou élégant. J'avais souvent vu cette ligne dans des exemples sans vraiment comprendre son utilité. En créant mes propres modules, j'ai réalisé son génie : un seul fichier peut jouer deux rôles — module réutilisable ET script autonome. Python distingue les deux contextes automatiquement via cette variable magique. C'est un design pattern que tout développeur Python professionnel applique systématiquement.

2. `os.path.join()` — la portabilité invisible. J'avais l'habitude d'écrire les chemins avec des `/` ou des \. Découvrir `os.path.join()` m'a révélé un principe fondamental : écrire du code qui fonctionne sur **tous les systèmes d'exploitation** sans modification. Python gère le séparateur approprié automatiquement. C'est le genre de détail qui sépare le code amateur du code professionnel.

3. La bibliothèque standard est immense. En explorant les modules disponibles, j'ai réalisé que Python livrait déjà des outils pour presque tout — compression de fichiers, emails, bases de données SQLite, serveurs HTTP, cryptographie, CSV, XML, JSON... Avant d'installer une bibliothèque externe, il vaut toujours la peine de vérifier si Python ne propose pas déjà ce qu'on cherche. La réponse est souvent oui.

Résumé du Chapitre 14

Ce que j'ai appris dans ce chapitre :

- `import module`, `from module import x`, `import module as alias`.
- **math** — constantes et fonctions mathématiques complètes.
- **random** — aléatoire, choix, mélange, graine reproductible.
- **os** — système de fichiers, chemins portables, variables d'environnement.
- **sys** — version Python, arguments CLI, flux standard.
- **datetime** — dates, heures, timedelta, formatage `strftime`.
- Créer ses propres modules — un fichier `.py` est déjà un module.
- `if __name__ == "__main__"` — pattern incontournable du code Python.
- **Packages** — organiser les modules dans des dossiers avec `__init__.py`.

"Ne réinventez pas la roue. Python a déjà une roue, un moteur, et une fusée — apprenez à les importer."